declaring that one model `belongs_to` another, you instruct Rails to maintain Primary Key-Foreign Key information between instances of the two models, and you also get a number of utility methods added to your model. Rails supports six types of associations:

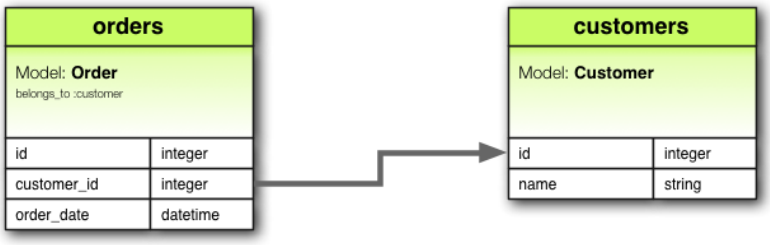
- `belongs_to`
- `has_one`
- `has_many`
- `has_many :through`
- `has_one :through`
- `has_and_belongs_to_many`

In the remainder of this guide, you'll learn how to declare and use the various forms of associations. But first, a quick introduction to the situations where each association type is appropriate.

## 2.1 The `belongs_to` Association

A `belongs_to` association sets up a one-to-one connection with another model, such that each instance of the declaring model "belongs to" one instance of the other model. For example, if your application includes customers and orders, and each order can be assigned to exactly one customer, you'd declare the order model this way:

```
class Order < ActiveRecord::Base
  belongs_to :customer
end
```




```
class Order < ActiveRecord::Base
  belongs_to :customer
end
```

`belongs_to` associations *must* use the singular term. If you used the pluralized form in the above example for the `customer` association in the `Order` model, you would be told that there was an "uninitialized constant Order::Customers". This is because Rails automatically infers the class name from the association name. If the association name is wrongly pluralized, then the inferred class will be wrongly pluralized too.

The corresponding migration might look like this:

```
class CreateOrders < ActiveRecord::Migration
  def change
    create_table :customers do |t|
      t.string :name
      t.timestamps null: false
    end

    create_table :orders do |t|
      t.belongs_to :customer, index: true
      t.datetime :order_date
      t.timestamps null: false
    end
  end
end
```